

Real-Time Examples of the Builder Design Pattern in C#

In this article, I will discuss the **Real-Time Examples of the Builder Design Pattern in C#**. Please read our previous article discussing the basic concepts of the [Builder Design Pattern in C#](#) with Examples. At the end of this article, you will understand the following Real-time Examples using the Builder Design Pattern in C#.

1. [Building a Complex Computer System](#)
2. [Building a Software for a Restaurant](#)
3. [Creating Complex Profile Objects for a User Registration System](#)
4. [Travel Agency where Users can Customize Holiday Packages](#)
5. [Creating a Configuration for a Server Deployment](#)
6. [StringBuilder](#)
7. [HttpRequestMessage](#)

Real-Time Example of Builder Design Pattern in C#: Building a Complex Computer System

Let's dive deep into a real-time example: building a complex computer system. A computer system might consist of various components like CPU, RAM, hard drive, graphics card, sound card, etc. Some components might be optional, while others are mandatory. Let us see how we can implement the above example using the Builder Design Pattern in C#:

```
using System;
namespace BuilderDesignPattern
{
    //Product
    public class Computer
    {
        public string CPU { get; set; }
        public string RAM { get; set; }
        public string HardDrive { get; set; }
        public string GraphicsCard { get; set; }
        public string SoundCard { get; set; }
        public void DisplaySpecifications()
        {
            Console.WriteLine($"CPU: {CPU}");
            Console.WriteLine($"RAM: {RAM}");
            Console.WriteLine($"Hard Drive: {HardDrive}");
            Console.WriteLine($"Graphics Card: {GraphicsCard} ?? "Not present"");
            Console.WriteLine($"Sound Card: {SoundCard} ?? "Not present"");
        }
    }
    //Builder (Abstract)
    public abstract class ComputerBuilder
    {
        protected Computer Computer { get; private set; } = new Computer();
        public abstract void SetCPU();
        public abstract void SetRAM();
        public abstract void SetHardDrive();
        public virtual void SetGraphicsCard() { } // Optional
        public virtual void SetSoundCard() { } // Optional
        public Computer GetComputer() => Computer;
    }
}
```

```

//Concrete Builder
public class GamingComputerBuilder : ComputerBuilder
{
    public override void SetCPU()
    {
        Computer.CPU = "High Performance CPU";
    }
    public override void SetRAM()
    {
        Computer.RAM = "16 GB DDR4";
    }
    public override void SetHardDrive()
    {
        Computer.HardDrive = "1 TB SSD";
    }
    public override void SetGraphicsCard()
    {
        Computer.GraphicsCard = "High-end Graphics Card";
    }
    public override void SetSoundCard()
    {
        Computer.SoundCard = "7.1 Surround Sound Card";
    }
}
//Director
public class ComputerShop
{
    public void ConstructComputer(ComputerBuilder builder)
    {
        builder.SetCPU();
        builder.SetRAM();
        builder.SetHardDrive();
        builder.SetGraphicsCard();
        builder.SetSoundCard();
    }
}

//Client Code
//Testing the Builder Design Pattern
public class Program
{
    public static void Main()
    {
        ComputerShop shop = new ComputerShop();
        ComputerBuilder builder = new GamingComputerBuilder();
        shop.ConstructComputer(builder);
        Computer computer = builder.GetComputer();
        computer.DisplaySpecifications();
        Console.ReadKey();
    }
}
}

```

In this example, if we want to add more computer configurations, like an "OfficeComputerBuilder," it can be done by creating another concrete builder. This way, the construction logic of a computer remains isolated from its representation, and it becomes easy to introduce new configurations without affecting existing code. When you run the above code, you will get the following output.

```
CPU: High Performance CPU
RAM: 16 GB DDR4
Hard Drive: 1 TB SSD
Graphics Card: High-end Graphics Card
Sound Card: 7.1 Surround Sound Card
```

Real-Time Example of Builder Design Pattern in C#: Building Software for a Restaurant

Suppose we're building software for a restaurant that allows customers to create a custom meal. Let us see how we can implement the above example using the Builder Design Pattern in C#:

```
using System;
namespace BuilderDesignPattern
{
    //Product: Custom Meal
    public class Meal
    {
        public string Drink { get; set; }
        public string MainDish { get; set; }
        public string Side { get; set; }
        public string Dessert { get; set; }
        public void ShowMeal()
        {
            Console.WriteLine($"Drink: {Drink}");
            Console.WriteLine($"Main Dish: {MainDish}");
            Console.WriteLine($"Side: {Side}");
            Console.WriteLine($"Dessert: {Dessert}");
        }
    }
    //Builder (Abstract Builder)
    public abstract class MealBuilder
    {
        protected Meal meal = new Meal();
        public abstract void SetDrink();
        public abstract void SetMainDish();
        public abstract void SetSide();
        public abstract void SetDessert();
        public Meal GetMeal() => meal;
    }
    //Concrete Builder: Vegan Meal
    public class VeganMealBuilder : MealBuilder
    {
        public override void SetDrink()
        {
            meal.Drink = "Water";
        }
        public override void SetMainDish()
        {
```

```

        meal.MainDish = "Vegan Burger";
    }
    public override void SetSide()
    {
        meal.Side = "Salad";
    }
    public override void SetDessert()
    {
        meal.Dessert = "Fruit Bowl";
    }
}
//Director: Meal Creator
public class MealCreator
{
    public void PrepareMeal(MealBuilder builder)
    {
        builder.SetDrink();
        builder.SetMainDish();
        builder.SetSide();
        builder.SetDessert();
    }
}

//Client Code
//Testing the Builder Design Pattern
public class Program
{
    public static void Main()
    {
        MealCreator mealCreator = new MealCreator();
        MealBuilder veganMealBuilder = new VeganMealBuilder();
        mealCreator.PrepareMeal(veganMealBuilder);
        Meal veganMeal = veganMealBuilder.GetMeal();
        Console.WriteLine("Vegan Meal:");
        veganMeal.ShowMeal();
        Console.ReadKey();
    }
}
}

```

In this scenario, the client wants a vegan meal. The MealCreator takes the VeganMealBuilder and uses it to create the meal. Suppose the restaurant decides to offer more types of meals in the future (e.g., "KetoMeal", "VegetarianMeal", etc.). In that case, we can easily extend the system by adding more concrete builders without altering the existing code, thus following the Open-Closed Principle. When you run the above code, you will get the following output.

```

Vegan Meal:
Drink: Water
Main Dish: Vegan Burger
Side: Salad
Dessert: Fruit Bowl

```

Real-Time Example of Builder Design Pattern in C#: Creating Complex Profile Objects for a User Registration System

Imagine an application where users can register. A user profile can contain basic information, address details, and settings preferences. Not every user provides all the details immediately, but the application should be able to build a profile step-by-step or based on the given input. Let us see how we can implement the above example using the Builder Design Pattern in C#:

```
using System;
namespace BuilderDesignPattern
{
    //Product: UserProfile
    public class UserProfile
    {
        public string FirstName { get; set; }
        public string LastName { get; set; }
        public string Email { get; set; }
        public string StreetAddress { get; set; }
        public string City { get; set; }
        public string State { get; set; }
        public string ZipCode { get; set; }
        public bool IsSubscribedToNewsletter { get; set; }
        public string ThemePreference { get; set; }
        public void DisplayProfile()
        {
            Console.WriteLine($"Name: {FirstName} {LastName}");
            Console.WriteLine($"Email: {Email}");
            Console.WriteLine($"Address: {StreetAddress}, {City}, {State}
{ZipCode}");
            Console.WriteLine($"Newsletter: {IsSubscribedToNewsletter}");
            Console.WriteLine($"Theme: {ThemePreference}");
        }
    }
    //Builder (Abstract Builder)
    public abstract class UserProfileBuilder
    {
        protected UserProfile UserProfile { get; private set; } = new
UserProfile();
        public abstract void SetBasicInfo(string firstName, string
lastName, string email);
        public abstract void SetAddress(string street, string city, string
state, string zip);
        public abstract void SetPreferences(bool newsletter, string theme);
        public UserProfile GetUserProfile() => UserProfile;
    }
    //Concrete Builder: ConcreteUserProfileBuilder
    public class ConcreteUserProfileBuilder : UserProfileBuilder
    {
        public override void SetBasicInfo(string firstName, string
lastName, string email)
        {
            UserProfile.FirstName = firstName;
            UserProfile.LastName = lastName;
            UserProfile.Email = email;
        }
    }
}
```

```

    }
    public override void SetAddress(string street, string city, string
state, string zip)
    {
        UserProfile.StreetAddress = street;
        UserProfile.City = city;
        UserProfile.State = state;
        UserProfile.ZipCode = zip;
    }
    public override void SetPreferences(bool newsletter, string theme)
    {
        UserProfile.IsSubscribedToNewsletter = newsletter;
        UserProfile.ThemePreference = theme;
    }
}
//Director: (Optional in this case, demonstrating without it)

//Client Code
//Testing the Builder Design Pattern
public class Program
{
    public static void Main()
    {
        var userProfileBuilder = new ConcreteUserProfileBuilder();
        userProfileBuilder.SetBasicInfo("John", "Doe",
"john.doe@example.com");
        userProfileBuilder.SetAddress("123 Main St", "Springfield",
"IL", "12345");
        userProfileBuilder.SetPreferences(true, "Dark");
        var userProfile = userProfileBuilder.GetUserProfile();
        userProfile.DisplayProfile();
        Console.ReadKey();
    }
}
}

```

In this example, based on the user's registration flow, we can decide which parts of the profile to construct at which point. If the user only provides basic information initially, we build that part and can later fill in the address and preferences as the user provides them. This is just a basic example, and in a real-world application, the Builder might include additional complexities and validations. When you run the above code, you will get the following output.

```

Name: John Doe
Email: john.doe@example.com
Address: 123 Main St, Springfield, IL 12345
Newsletter: True
Theme: Dark

```

Real-Time Example of Builder Design Pattern in C#: Travel Agency where Users can Customize Holiday Packages

Let's consider a scenario for a travel agency where users can customize holiday packages. A typical holiday package might include flights, hotel bookings, car rentals, and excursions. Not every user will select all components,

but the package should be flexible to accommodate various choices. Let us see how we can implement the above example using the Builder Design Pattern in C#:

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace BuilderDesignPattern
{
    //Product: HolidayPackage
    public class HolidayPackage
    {
        public string Flight { get; set; }
        public string Hotel { get; set; }
        public string CarRental { get; set; }
        public List<string> Excursions { get; private set; } = new
List<string>();
        public void DisplayPackageDetails()
        {
            Console.WriteLine($"Flight: {Flight ?? "Not selected"}");
            Console.WriteLine($"Hotel: {Hotel ?? "Not selected"}");
            Console.WriteLine($"Car Rental: {CarRental ?? "Not
selected"}");
            Console.WriteLine("Excursions: " + (Excursions.Any() ?
string.Join(", ", Excursions) : "No excursions selected"));
        }
    }
    //Builder (Abstract Builder)
    public abstract class HolidayPackageBuilder
    {
        protected HolidayPackage Package { get; private set; } = new
HolidayPackage();
        public abstract void BookFlight(string flightDetails);
        public abstract void BookHotel(string hotelName);
        public abstract void RentCar(string carDetails);
        public abstract void AddExcursion(string excursion);
        public HolidayPackage GetPackage() => Package;
    }
    //Concrete Builder: CustomHolidayPackageBuilder
    public class CustomHolidayPackageBuilder : HolidayPackageBuilder
    {
        public override void BookFlight(string flightDetails)
        {
            Package.Flight = flightDetails;
        }
        public override void BookHotel(string hotelName)
        {
            Package.Hotel = hotelName;
        }
        public override void RentCar(string carDetails)
        {
            Package.CarRental = carDetails;
        }
        public override void AddExcursion(string excursion)
        {

```

```

        Package.Excursions.Add(excursion);
    }
}
//Director: TravelAgent
public class TravelAgent
{
    public void CreatePackage(HolidayPackageBuilder builder, bool
wantsFlight, bool wantsHotel, bool wantsCar, IEnumerable<string>
excursions)
    {
        if (wantsFlight) builder.BookFlight("Flight details...");
        if (wantsHotel) builder.BookHotel("Fancy Hotel");
        if (wantsCar) builder.RentCar("SUV Model XYZ");
        foreach (var excursion in excursions)
        {
            builder.AddExcursion(excursion);
        }
    }
}
//Client Code
//Testing the Builder Design Pattern
public class Program
{
    public static void Main()
    {
        var travelAgent = new TravelAgent();
        var packageBuilder = new CustomHolidayPackageBuilder();
        travelAgent.CreatePackage(packageBuilder, true, true, false,
new[] { "Beach trip", "Mountain hiking" });
        var holidayPackage = packageBuilder.GetPackage();
        holidayPackage.DisplayPackageDetails();
        Console.ReadKey();
    }
}
}

```

In this real-time scenario, the user can customize their holiday package based on their preferences. Using the Builder pattern, the TravelAgent can easily construct a HolidayPackage tailored to the user's needs without overcomplicating the process. When you run the above code, you will get the following output.

```

Flight: Flight details...
Hotel: Fancy Hotel
Car Rental: Not selected
Excursions: Beach trip, Mountain hiking

```

Real-Time Example of Builder Design Pattern in C#: Creating a Configuration for a Server Deployment

An organization's IT department frequently deploys servers. However, not every server is the same. Some servers are web servers, some are database servers, and others might be cache servers. Each type requires a different configuration, software, and security measures. The Builder pattern can streamline the server deployment by ensuring valid and complete configurations. Let us see how we can implement the above example using the Builder Design Pattern in C#:


```

using System;
using System.Collections.Generic;
using System.Linq;
namespace BuilderDesignPattern
{
    //Product: ServerConfiguration
    public class ServerConfiguration
    {
        public string OperatingSystem { get; set; }
        public string ServerType { get; set; } // e.g., Web, Database,
Cache
        public string Database { get; set; } // Optional: Database
software
        public List<string> Middleware { get; set; } = new List<string>();
        public bool HasFirewall { get; set; }
        public void DisplayConfig()
        {
            Console.WriteLine($"OS: {OperatingSystem}");
            Console.WriteLine($"Server Type: {ServerType}");
            Console.WriteLine($"Database: {Database ?? "None"}");
            Console.WriteLine($"Middleware: {string.Join(", ",
Middleware)}}");
            Console.WriteLine($"Firewall: {(HasFirewall ? "Enabled" :
"Disabled"}}");
        }
    }
    //Builder (Abstract Builder)
    public abstract class ServerBuilder
    {
        protected ServerConfiguration Configuration { get; private set; } =
new ServerConfiguration();
        public abstract void SetOperatingSystem();
        public abstract void SetServerType();
        public abstract void InstallDatabase();
        public abstract void InstallMiddleware();
        public abstract void EnableFirewall();
        public ServerConfiguration GetConfiguration() => Configuration;
    }
    //Concrete Builder: WebServerBuilder
    public class WebServerBuilder : ServerBuilder
    {
        public override void SetOperatingSystem()
        {
            Configuration.OperatingSystem = "Linux";
        }
        public override void SetServerType()
        {
            Configuration.ServerType = "Web";
        }
        public override void InstallDatabase()
        {
            // For this web server, no DB installation
        }
        public override void InstallMiddleware()
    }
}

```

```

        {
            Configuration.Middleware.Add("Nginx");
            Configuration.Middleware.Add("Node.js");
        }
        public override void EnableFirewall()
        {
            Configuration.HasFirewall = true;
        }
    }
    //Director: ServerDeployment
    public class ServerDeployment
    {
        public void Deploy(ServerBuilder builder)
        {
            builder.SetOperatingSystem();
            builder.SetServerType();
            builder.InstallDatabase();
            builder.InstallMiddleware();
            builder.EnableFirewall();
        }
    }

    //Client Code
    //Testing the Builder Design Pattern
    public class Program
    {
        public static void Main()
        {
            var deployer = new ServerDeployment();
            var webServerBuilder = new WebServerBuilder();
            deployer.Deploy(webServerBuilder);
            var serverConfig = webServerBuilder.GetConfiguration();
            serverConfig.DisplayConfig();
            Console.ReadKey();
        }
    }
}

```

In this example, deploying a new web server configuration is streamlined. If the IT department needs to introduce a new server type, like a “DatabaseServerBuilder,” it’s straightforward without changing the ServerDeployment director. This ensures consistency and reduces the chances of deployment errors. When you run the above code, you will get the following output.

```

OS: Linux
Server Type: Web
Database: None
Middleware: Nginx, Node.js
Firewall: Enabled

```

Real-Time Example of Builder Design Pattern in C#: StringBuilder

The StringBuilder class in C# is itself a representation of the Builder design pattern. Its primary purpose is to help construct or manipulate strings efficiently. When a string needs to be modified multiple times, a StringBuilder can be

much more efficient than a regular string, which is immutable in C#.

Let's build upon this idea and create a StringBuilder-like class using the Builder pattern principles, focusing on its design and utility. Let us see how we can implement the above example using the Builder Design Pattern in C#.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
namespace BuilderDesignPattern
{
    //Product: CustomString
    public class CustomString
    {
        private StringBuilder _stringBuilder = new StringBuilder();
        public void Append(string str)
        {
            _stringBuilder.Append(str);
        }
        public override string ToString()
        {
            return _stringBuilder.ToString();
        }
    }
    //Builder
    public class StringConstructor
    {
        private CustomString _customString = new CustomString();
        public StringConstructor AppendLine(string line)
        {
            _customString.Append(line + Environment.NewLine);
            return this;
        }
        public StringConstructor AppendSpace(string word)
        {
            _customString.Append(word + " ");
            return this;
        }
        public StringConstructor AppendWords(params string[] words)
        {
            foreach (var word in words)
            {
                AppendSpace(word);
            }
            return this;
        }
        public CustomString Build()
        {
            return _customString;
        }
    }

    //Client Code
    //Testing the Builder Design Pattern
}
```

```

public class Program
{
    public static void Main()
    {
        var constructor = new StringConstructor();
        constructor.AppendLine("Hello, Builder pattern!")
            .AppendWords("This", "is", "a", "custom", "string",
"builder!");

        .AppendLine("It's efficient.");
        var customStr = constructor.Build();
        Console.WriteLine(customStr);
        Console.ReadKey();
    }
}

```

In this “real-time” example, we’ve essentially constructed a simplified `StringBuilder`. We’ve included a few methods for demonstration purposes, such as `AppendLine`, `AppendSpace`, and `AppendWords`. These methods return the builder itself (`StringConstructor`), allowing for method chaining. The `Build` method then returns the final constructed product (`CustomString`).

This structure can easily be extended to include other methods for more advanced string manipulations, illustrating how the Builder pattern offers a clean and structured way to create and configure objects. When you run the above code, you will get the following output.

```

Hello, Builder pattern!
This is a custom string builder! It's efficient.

```

Real-Time Example of Builder Design Pattern in C#: `HttpRequestMessage`

The `HttpRequestMessage` in the `System.Net.Http` namespace is a common class used to construct HTTP requests in C#. Its usage can benefit from the Builder pattern, especially when creating more complex requests with various headers, content, and configurations. Let’s design a builder for `HttpRequestMessage`. Let us see how we can implement the above example using the Builder Design Pattern in C#:

```

using System;
using System.Collections.Generic;
using System.Net.Http;
using System.Text;
namespace BuilderDesignPattern
{
    //Builder Interface
    public interface IHttpRequestBuilder
    {
        IHttpRequestBuilder WithMethod(HttpMethod method);
        IHttpRequestBuilder WithUri(Uri uri);
        IHttpRequestBuilder WithContent(HttpContent content);
        IHttpRequestBuilder WithHeader(string name, string value);
        HttpRequestMessage Build();
    }
    // Concrete Builder
    public class HttpRequestBuilder : IHttpRequestBuilder
    {
        private HttpMethod _method = null;
        private Uri _uri = null;

```

```

        private HttpContent _content = null;
        private Dictionary<string, string> _headers = new
Dictionary<string, string>();
        public IHttpRequestBuilder WithMethod(HttpMethod method)
        {
            _method = method;
            return this;
        }
        public IHttpRequestBuilder WithUri(Uri uri)
        {
            _uri = uri;
            return this;
        }
        public IHttpRequestBuilder WithContent(HttpContent content)
        {
            _content = content;
            return this;
        }
        public IHttpRequestBuilder WithHeader(string name, string value)
        {
            _headers[name] = value;
            return this;
        }
        public HttpRequestMessage Build()
        {
            var message = new HttpRequestMessage(_method, _uri)
            {
                Content = _content
            };
            foreach (var header in _headers)
            {
                message.Headers.Add(header.Key, header.Value);
            }
            return message;
        }
    }
    //Client Code
    //Testing the Builder Design Pattern
    public class Program
    {
        public static void Main()
        {
            var request = new HttpRequestBuilder()
                .WithMethod(HttpMethod.Post)
                .WithUri(new Uri("https://api.example.com/items"))
                .WithContent(new StringContent("{ \"name\": \"sampleItem\" }",
Encoding.UTF8, "application/json"))
                .WithHeader("Authorization", "Bearer my_token")
                .Build();
            // You can now use this request with HttpClient to send the
actual request.
            using (var client = new HttpClient())
            {
                var response = client.SendAsync(request).Result;
            }
        }
    }

```

```

        Console.WriteLine(response.StatusCode);
    }

    Console.ReadKey();
}
}
}

```

In this example, the `HttpRequestBuilder` follows the Builder pattern, allowing for a fluent and intuitive configuration of the `HttpRequestMessage`. It provides clarity, especially when working with more complex HTTP requests.

Advantages of Builder Design Pattern in C#:

The Builder Design Pattern offers several advantages in C# programming, especially when dealing with constructing complex objects. Some of the key benefits include:

- **Separation of Construction and Representation:** The Builder pattern separates the construction process of an object from its representation, allowing the same construction process to create different representations. This separation enhances the modularity of the code.
- **Encapsulation of Complex Creation Logic:** The pattern encapsulates complex object creation logic within the builder, making the client code cleaner and simpler. This is particularly useful when creating objects with numerous initialization parameters, some of which may be optional.
- **Improved Readability and Maintainability:** Using the Builder pattern, you can avoid telescoping constructor anti-patterns (where a constructor may have many parameters, not all of which are always needed). This improves code readability and maintainability.
- **Immutability of Objects:** The Builder pattern can be used to build immutable objects (objects whose state cannot change after they are constructed). Immutable objects are simpler to understand and work with and are also thread-safe.
- **Control Over Object Construction Process:** The pattern provides more control over the construction process than direct instantiation. It allows building an object step-by-step, and the product is only returned when it's entirely ready.
- **Fluent Interface and Method Chaining:** Many builder implementations use fluent interfaces and method chaining, making client code more readable and easily understood.
- **Flexibility in Object Instantiation:** Builders can construct complex objects that would require a lot of setup code to create otherwise. They can also instantiate subclasses of an immutable class.
- **Ease of Adding New Parameters:** Adding new parameters to the object's constructor through the builder is easy and does not require changing the constructor signature, thus not affecting existing client code.

Disadvantages of Builder Design Pattern in C#:

The Builder Design Pattern, while offering numerous benefits for creating complex objects in a structured and clear manner, also has its own set of disadvantages, particularly in the context of C# development:

- **Increased Complexity:** The Builder pattern introduces additional layers and objects (like the Director and Concrete Builder), which can overcomplicate the code, especially for simple objects that might not require such an elaborate construction process.
- **Duplication of Code:** There might be a code duplication, as the properties being set in the builder essentially repeat what exists in the object being constructed.
- **Tedious to Implement:** Implementing the Builder pattern can be more tedious and verbose than using a simple constructor or a factory method, especially for objects with many parameters.
- **Efficiency Concerns:** Building an object step by step can be less efficient, especially if the built product is complex. This can have performance implications in scenarios where many objects are being created.
- **Maintenance Overhead:** Maintaining the Builder pattern can be challenging, particularly if changes are frequently made to the underlying object's structure, as these changes might need to be reflected in the corresponding builder class.
- **Immutability Complications:** While the Builder pattern is great for creating immutable objects, ensuring immutability can sometimes be challenging, especially if the built object requires post-construction modifications.
- **Understanding and Learning Curve:** For developers unfamiliar with the pattern, it can introduce a learning curve and make the code harder to understand at first glance than straightforward object construction.

techniques.

- **Possible Misuse:** The Builder pattern might be misused when simpler object creation methods would suffice, leading to unnecessary abstraction and complexity.

In the next article, I will discuss the [Fluent Interface Design Pattern in C#](#) with some examples. In this article, I explain Real-time Examples of the Builder Design Pattern using C#. I hope you understand the need and use of the Builder Design Pattern in real-time examples in C#.